Step 2 — The Grounded Answer — Solution

> Back to the step · Hint 1 · Hint 2

Story: this asks the model to answer the question using **only** the retrieved code context built in Step 1, never its own general training knowledge. **If not:** the model answers from its training memory, which is generic and often flat-out wrong for *this specific* codebase — the whole reason you bothered retrieving real snippets would be lost.

The answer chain

```
# src/answer.py
# Answer the question using ONLY the retrieved code context, so the answer is
# correct for THIS codebase - not the model's general knowledge.
from langchain_core.prompts import ChatPromptTemplate

ANSWER_PROMPT = (
    "You answer questions about a codebase using ONLY the code below. If the "
    "answer is not in it, say you don't know. Be specific and name the "
    "functions or files involved.\n\nCode:\n{context}\n\nQuestion: {question}"
)


def answer_question(model, question, context):
    prompt = ChatPromptTemplate.from_template(ANSWER_PROMPT)
    chain = prompt | model
    reply = chain.invoke({"context": context, "question": question})
    return reply.content
```

Why grounding is the whole point

- **"Using ONLY the code below"** is the instruction that keeps the answer tied to the repo. Without it, the model falls back on how `login` is *usually* written — which may be nothing like how *this* repo does it.
- **"If it's not in it, say you don't know"** is the honesty rule: a question about a feature the repo doesn't have should get "I don't know", not a plausible invention. Same discipline as every RAG answer (Doc08/10).

- **"Name the functions or files involved"** nudges the model to point at specifics, which pairs with the cited-files list Step 1 already built — the user gets both a written answer and the exact places to look.
- The context handed in is Step 1's `build_context(snippets)` — one block with `# File:` headers — so the model can see which snippet came from which file as it answers.

Tests for this step: `answer_question(model, "what does connect do?", context)` on a hand-written `context` returns a string. The full grounded behavior — a real question answered from retrieved code, "I don't know" on an off-topic one — is the integration test in Step 3.